Full-Stack Developer Test Assignment: TradeLocker API Integration

Feel free to use AI tools during development. We are not evaluating whether the code was AI-assisted, but whether you can understand the implementation, navigate the codebase, explain your technical decisions, and discuss the trade-offs of your solution.

Please also avoid overengineering or spending excessive time on the assignment. The goal is to demonstrate your ability to deliver a small but complete full-stack application. Focus on correctness, clarity, and maintainability rather than adding a large number of features.

We also expect regular Git commits throughout development. Please do not commit the entire project in a single commit; we would like to see a reasonable development progression and commit history.

Goal

Build a small full-stack application that integrates with the **TradeLocker Public API**.

The goal is not to build a production trading platform, but to demonstrate that you can deliver a working full-stack app with:

- frontend
- backend REST API
- database persistence
- external API integration
- authentication/session handling
- logging
- Docker-based setup
- clean GitHub delivery

TradeLocker API documentation:

https://public-api.tradelocker.com/docs/getting-started

Context for Candidate

You will receive a discount code to create a free TradeLocker account through the E8 Markets portal. This account should be used for testing the integration.

Technical Requirements

Frontend

Use either:

- React
- Vue

The frontend should provide a simple but usable interface for:

- logging in
- selecting a trading account if multiple accounts are available
- selecting a tradable symbol/instrument
- viewing account information
- viewing current position state/history

UI design does not need to be polished, but the app should be understandable and usable.

Backend

Backend technology is candidate's choice.

Examples:

- Node.js
- Python
- Go
- Java
- C#
- PHP

The backend must be responsible for all TradeLocker API communication.

The frontend must **not** call TradeLocker directly.

Database

Use any database suitable for a small app.

Examples:

- PostgreSQL
- MySQL
- SQLite
- MongoDB

The app must persist:

- login/session-related state where appropriate
- position state history
- backend logs or API interaction logs

Important Restriction

Do **not** use a premade TradeLocker API SDK/client package.

For example, do not use:

- `tradelocker-python`
- any equivalent TradeLocker wrapper/client library

Basic HTTP, REST, JSON, authentication, database, and framework libraries are allowed.

Examples of allowed libraries:

- axios
- fetch
- requests
- httpx
- express
- fastapi
- ORM/query builders
- validation libraries

- logging libraries

The point is to see how you structure the integration yourself.

Must-Have Features

1. TradeLocker Login

Implement login using TradeLocker credentials.

The backend should handle:

- authentication
- access token storage/handling
- token expiration awareness
- safe error handling for failed login

2. Login State Management

The frontend should maintain login state.

Minimum expectation:

- user can log in
- app knows whether user is authenticated
- user can log out
- protected views are not accessible before login

3. Account Information

Display account details such as:

- balance
- equity
- margin-related data if available
- account ID / account number

4. Tradable Symbol Selection

Fetch and display available tradable instruments/symbols.

The user should be able to:

- browse/search symbols
- select a symbol
- see basic symbol information

5. Position State History

Display current and/or historical position state.

The app should store snapshots of position state in the database.

Example fields:

- position ID
- symbol
- side
- size
- open price
- current price
- PnL
- timestamp

A simple polling mechanism or manual refresh is acceptable.

6. Backend Logging

Implement some form of backend logging.

Minimum:

- login attempts
- TradeLocker API requests
- failed API calls
- sync operations

Logs may be stored in:

- log file
- structured console output

7. Docker Support

The project must be runnable using Docker.

Expected:

```shell
docker compose up --build
```

The setup should include:

- frontend container
- backend container
- database container (unless using SQLite)

The project should use environment variables for configuration.

8. GitHub Delivery

Deliver the project as a GitHub repository.

The repository must include:

- README
- setup instructions
- `.env.example`
- Docker instructions
- brief architecture explanation
- known limitations / assumptions

Could-Have Features

These are optional but considered a plus.

Historical Data Chart

Display historical price data for the selected symbol in a chart.

Examples:

- candlestick chart
- line chart

Token Refresh

Implement refresh-token flow instead of forcing re-login.

Background Sync

Add a backend job that periodically syncs:

- account info
- position state
- closed trades

Better Error UX

Show clear frontend messages for:

- invalid login
- expired session
- API rate limits
- missing account
- TradeLocker API failures

Tests

Add unit or integration tests for:

- backend API routes
- TradeLocker API service layer

- database persistence
- frontend critical flows

Suggested App Flow

1. User opens app
2. User logs in with TradeLocker credentials
3. Backend authenticates with TradeLocker
4. App fetches available accounts
5. User selects account
6. App displays account information
7. App fetches tradable symbols
8. User selects symbol
9. App displays current position state
10. User can sync/store position state history
11. User can sync/store closed trades history
12. Frontend displays stored historical data from the app database

Evaluation Criteria

We will evaluate:

Functionality

- app runs successfully
- login works
- account info is displayed
- symbols can be selected
- position state history is stored and displayed
- API calls go through the backend only

Code Quality

- clear project structure
- readable code
- sensible abstractions
- maintainable API integration layer
- no unnecessary overengineering

Expected Time Investment

This should be a relatively small full-stack project, not a large production system.

Focus on:

- correctness
- completeness
- clean implementation
- maintainability

rather than visual polish or advanced architecture.

Submission

Submit:

- GitHub repository URL
- short notes about implementation decisions
- setup instructions
- known limitations
- optional screenshots or demo video